

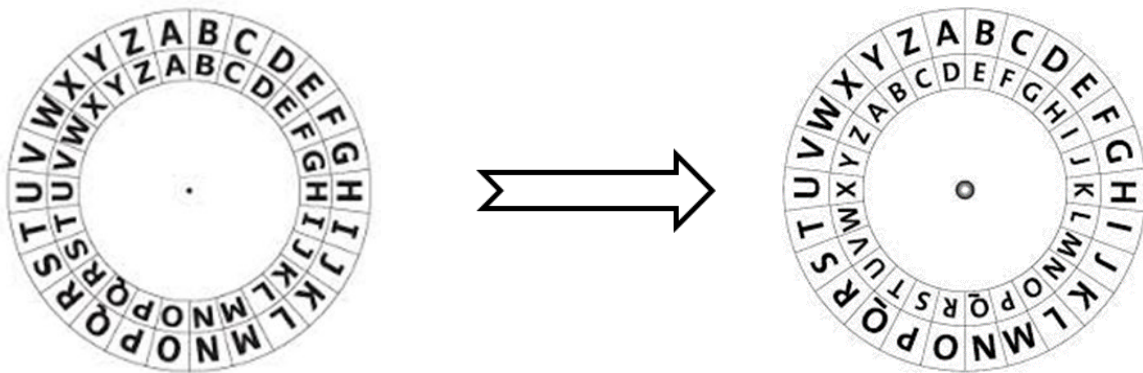
Lab Exercise 01: Caesar Cipher

BACKGROUND:

We examined Caesar Algorithm to encrypt and decrypt messages in the class. In this Lab, we will develop a program that implements the Caesar Algorithm. You will use your class notes for the details of the algorithm.

PREPARATION:

1. Discuss what input(s) and output(s) should be considered if we want to encrypt a message.
2. The cipher wheel was used by Caesar to encrypt each character in the message as we discussed in the classroom. For example, if the key is 3, Caesar turns the inner disk three times anticlockwise:



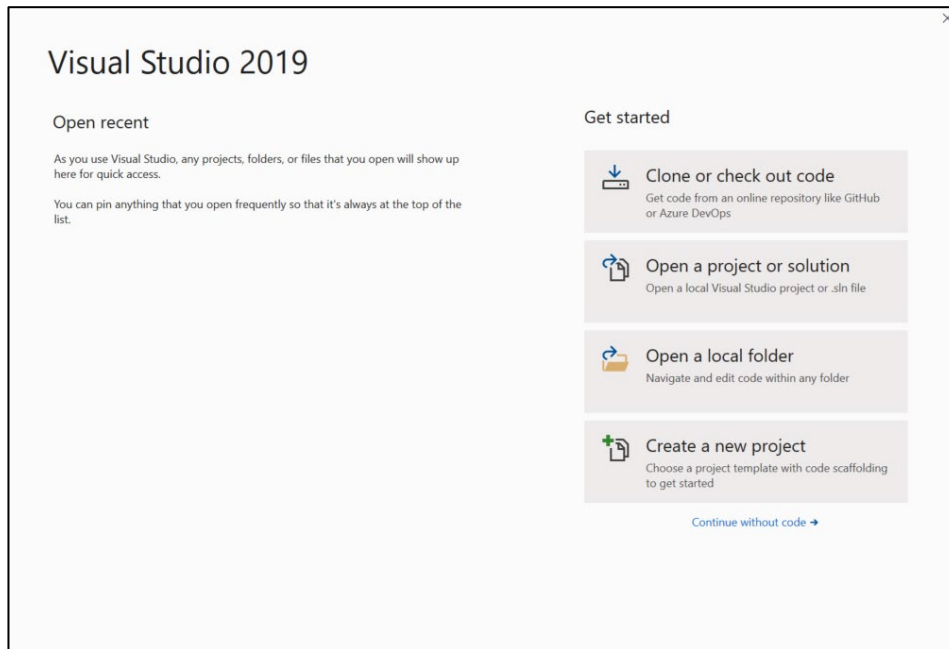
Design pseudocode for Caesar Encryption using a cipher wheel.

Note: Pseudocode is a high level description of the algorithm. It is for human reading, not machine reading.

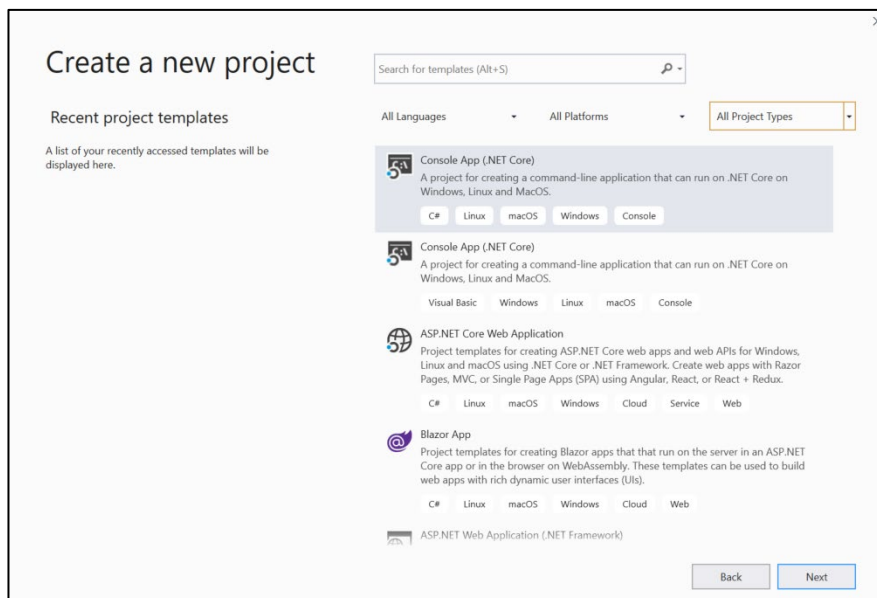
CREATING a NEW CONSOLE APPLICATION using VISUAL STUDIO 2019:

1. Run Visual Studio 2019. This should create a specific folder:

C:\Users\<Your ID>\source\repos

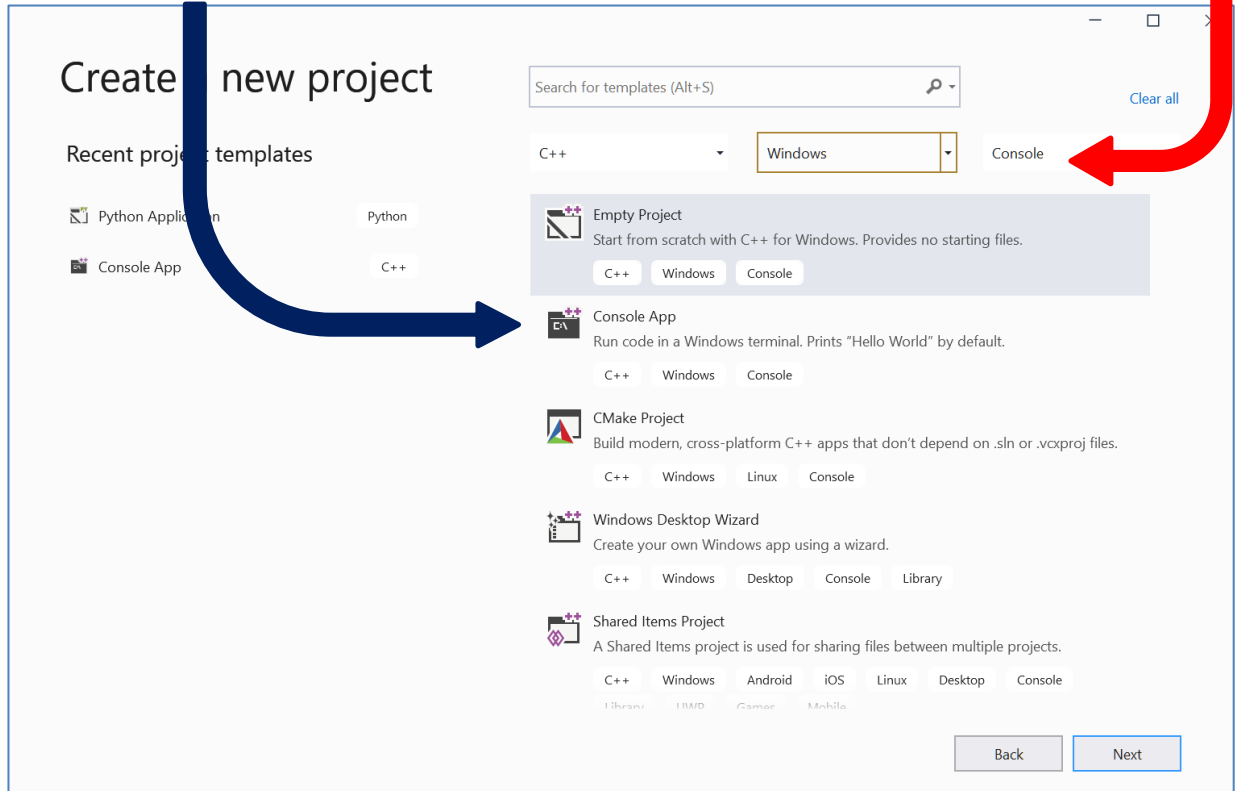


2. Click on Create New Project. This will bring the following window (The view may be different depending on the version):



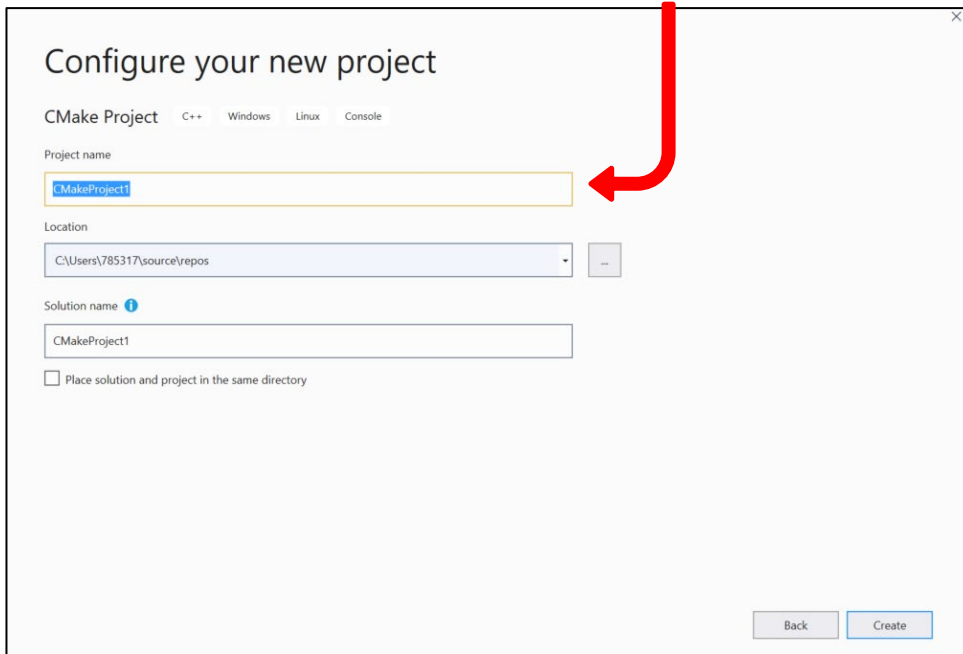
3. Select C++, Windows and Console in All Languages, All Platforms and All Project Types tabs. This will change the options at the right hand side pane (This view may be different depending on Full or Community versions).

Select CLR Console App in this pane and click on Next:



Communication and Security Protocols

4. In the new window, change the Project Name to CaesarCipher and click on Create.



Configure your new project

CMake Project C++ Windows Linux Console

Project name
CMakeProject1

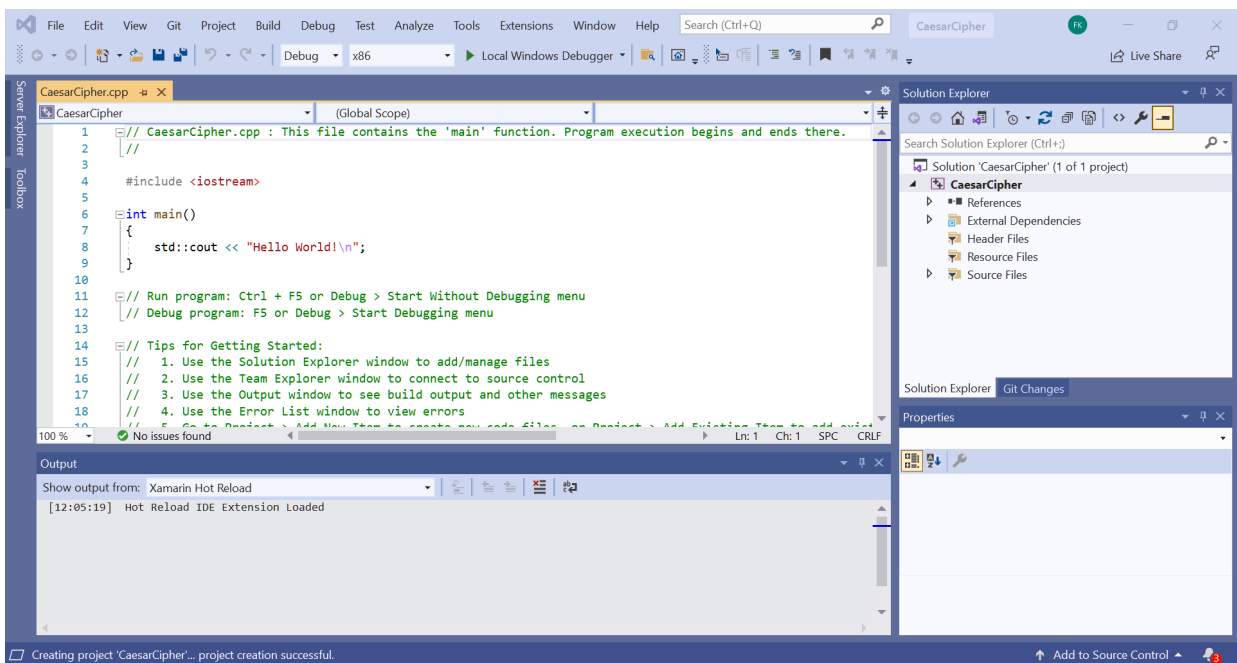
Location
C:\Users\785317\source\repos

Solution name
CMakeProject1

☐ Place solution and project in the same directory

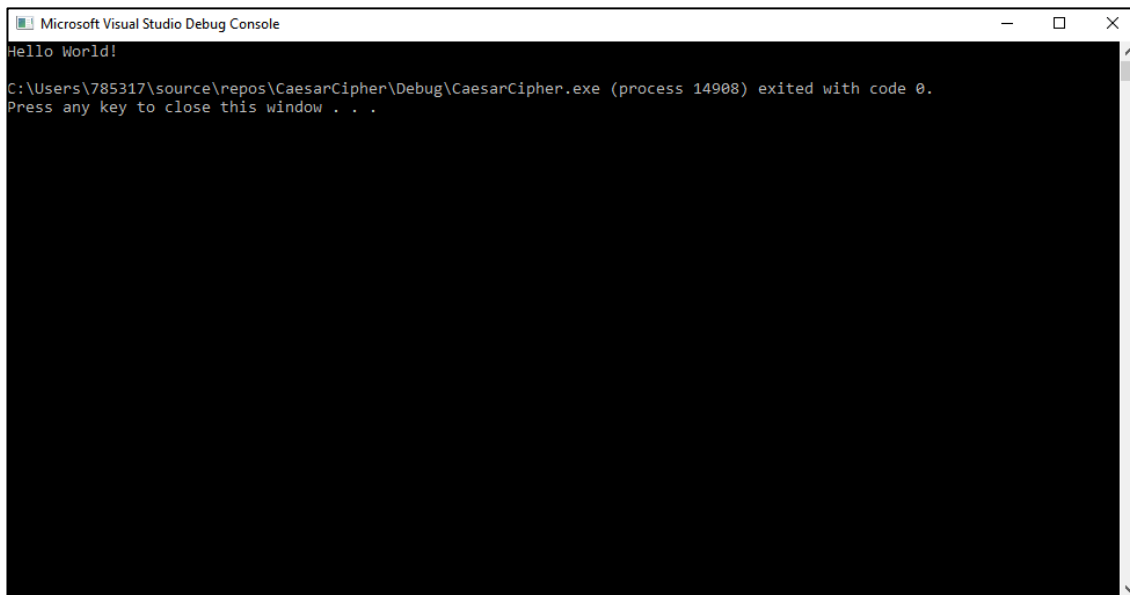
Back Create

5. This will create the project with a default cpp file :



Communication and Security Protocols

6. Run the program using either CTRL + F5 or Debug > Start Without Debugging menu. You should see the greeting message :



```
Microsoft Visual Studio Debug Console
Hello World!
C:\Users\785317\source\repos\CaesarCipher\Debug\CaesarCipher.exe (process 14908) exited with code 0.
Press any key to close this window . . .
```

CAESAR ENCRYPTION:

I. INPUT and OUTPUT

For the sake of simplicity the message to be encrypted consists of only English UPPERCASE characters (A-Z) without any punctuation and blank spaces.

It is a good practice to start with the input and output structure of your program before concentrate on the key functionalities.

What are the inputs and the output of a message encryption system?

What data types should be used in declaring variables for them?

1. We will use the standard C++ library. It is better to use its namespace `std` to avoid `std::` for each element of this library. Then add `using namespace std;` before `int main()`.
2. Delete the line `std::cout << "Hello World!\n";` in the `CaesarCipher.cpp` file.
3. Add appropriate variables to `main()` for the `plainText` (string) and the `key` (integer).
Tip: C++ has no built-in string variable type. You need to include a header file "string.h" in the source code by using `#include <string>`. Then you can create string variables using the `string` keyword.
4. Ask the user what plaintext and key are.

HINTS:

a. use a `cout` object to display a message :

```
cout << "Enter a MESSAGE to encrypt (Use UPPERCASE Characters): " << endl;
```

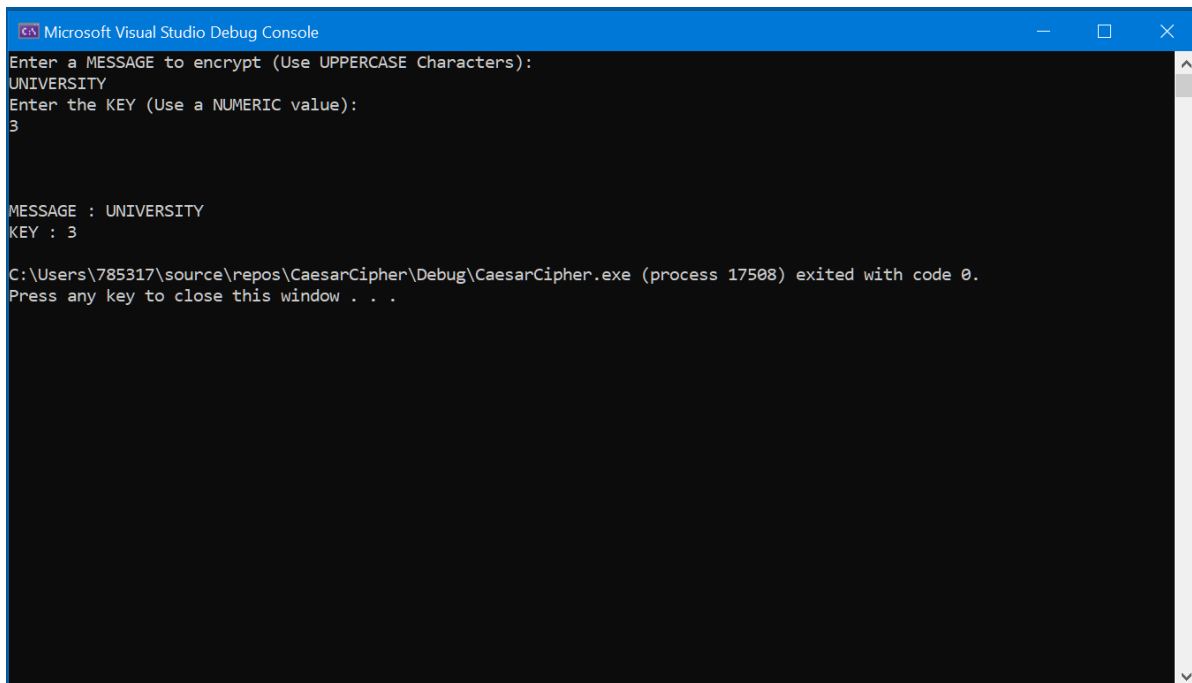
b. use `cin` object to read the message to be encrypted from the console :

```
cin >> plainText;  
cin.clear();
```

c. use `cin` object to read the key (before this better to display a message saying "Enter the KEY (Use a NUMERIC value):"):

```
cin >> key;  
cin.clear();
```

5. Display both variables using a `cout` object.
6. Run the program using either `CTRL + F5` or `Debug > Start Without Debugging` menu :



```
Microsoft Visual Studio Debug Console
Enter a MESSAGE to encrypt (Use UPPERCASE Characters):
UNIVERSITY
Enter the KEY (Use a NUMERIC value):
3

MESSAGE : UNIVERSITY
KEY : 3

C:\Users\785317\source\repos\CaesarCipher\Debug\CaesarCipher.exe (process 17508) exited with code 0.
Press any key to close this window . . .
```

II. CREATING ALPHABET and SHIFTING IT ACCORDING TO KEY

Before starting this task, it is better to suppress the lines for displaying `plainText` and `key` with `//` as we do not display them anymore.

It is better to practice using modular programming by dividing the program into small chunks and implement them using functions.

The first task in a Caesar Cipher is to rotate the alphabet according to the given key. Let's create a function named `shiftAlphabet()` for this aim:

The function will take the current alphabet and the key as inputs and returns a shifted alphabet.

First, we have to declare `alphabet` and `shiftedAlphabet` variables in `main()`. They should hold the characters of the English alphabet so they should be declared as a string in C++. We know the content of the `alphabet` so we can initialize it during the declaration:

```
string alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
```

We know that `shiftedAlphabet` will hold shifted English alphabet having 26 characters. As we do not know the order of them at the beginning, we can initialise this string object with space characters.

Communication and Security Protocols

```
string shiftedAlphabet(26, ' ');
```

The `shiftAlphabet()` function should take the original alphabet and the key as parameters, and should return the shifted alphabet. It should be called as follows in `main()` :

```
shiftedAlphabet = shiftAlphabet(alphabet, key);
```

The function `shiftAlphabet` should be implemented outside of `main()`. Add the following **before** `main()` :

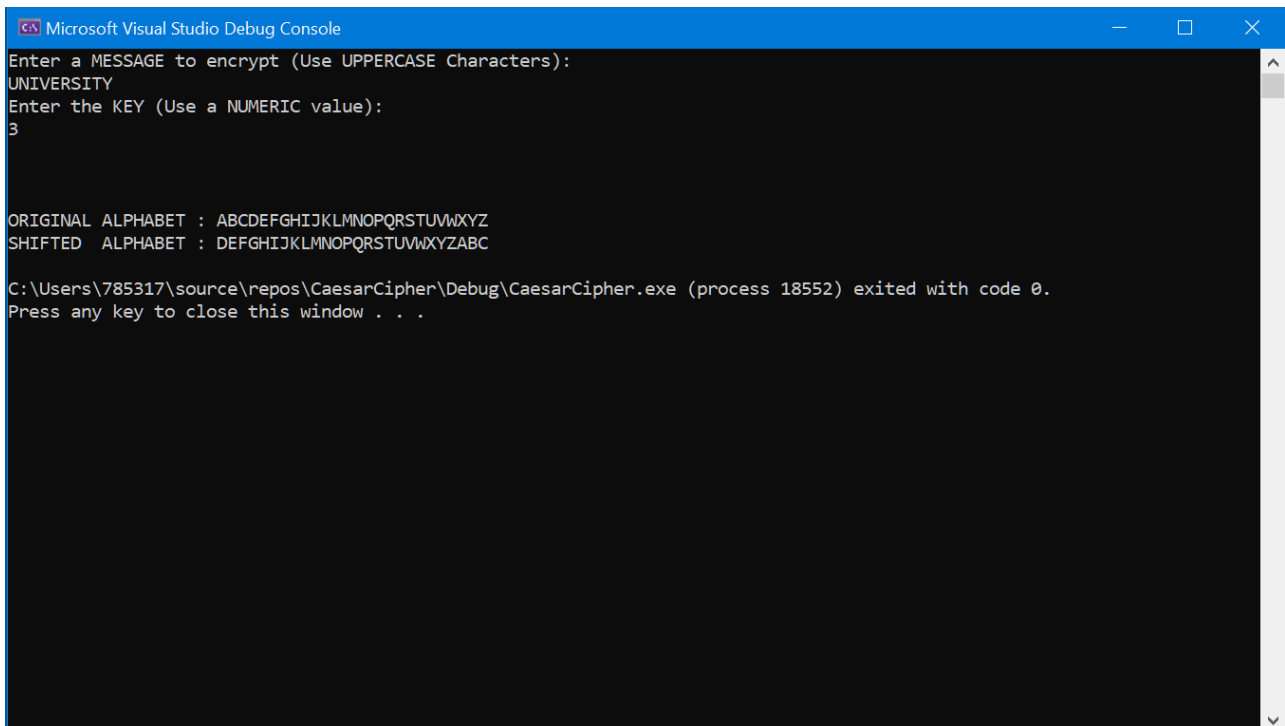
```
string shiftAlphabet(string alphabet, int key)
{
}
}
```

Now, we need to implement an algorithm that copies the alphabet characters to `shiftedAlphabet` by rotating them by using `key`.

1. Design a simple algorithm for this and code it inside the `shiftAlphabet()` function.

- Hint: -- Declare a string named `shiftedAlphabet`. This will be a local variable storing shifted alphabet. The size of this variable should be equal to the size of `alphabet`. You can obtain the size using the `length()` method of `alphabet`.
- Use for loops to iterate `alphabet` and copy the characters to `shiftedAlphabet` considering the value of `key`.
 - Return `shiftedAlphabet` to the main using a `return` command.

2. Test your method by printing the values of `alphabet` and `shiftedAlphabet` in the `main()` after the function call. The output of your programme should be following:



```
Microsoft Visual Studio Debug Console
Enter a MESSAGE to encrypt (Use UPPERCASE Characters):
UNIVERSITY
Enter the KEY (Use a NUMERIC value):
3

ORIGINAL ALPHABET : ABCDEFGHIJKLMNOPQRSTUVWXYZ
SHIFTED ALPHABET : DEFGHIJKLMNOPQRSTUVWXYZABC

C:\Users\785317\source\repos\CaesarCipher\Debug\CaesarCipher.exe (process 18552) exited with code 0.
Press any key to close this window . . .
```

3. Run the program with different key values and check how the alphabet is shifted.
What happens if you use values more than 27? Why?

III. DESIGNING ENCRYPTION FUNCTION

Again, to implement Caesar encryption it is better to use a function. Let's name it `encryptMessage()`.

This function will get `plaintext` and the `shiftedAlphabet` and then constructs and returns `ciphertext`.

It should be called from `main()` as follows :

```
cipherText = encryptMessage(plainText, shiftedAlphabet);
```

1. Design a function for `encryptMessage()`.

Where should this function be created?

What are the input and output parameters?

2. Define a string variable named `cipherText`. The length of this should be equal to that of `plainText`. How can you create a string with a length?

TIP: How did we define `shiftedAlphabet` in the `main()`?

3. Design a simple algorithm to generate `ciphertext` using `shiftedAlphabet`

Hints: - Use a `for` loop to iterate `plainText` until reaching the length of the `plainText`. Remember the length of a string object can be obtained using the `length()` method.

- In each iteration extract the current character of `plainText`.

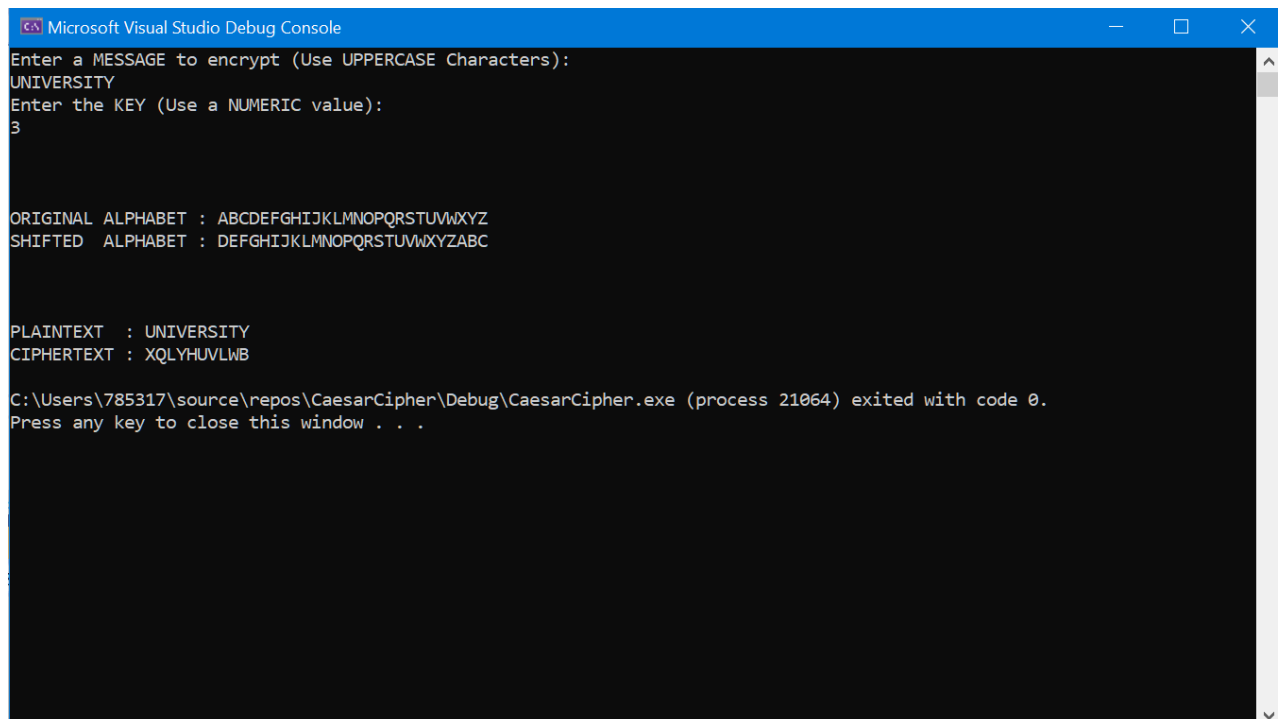
- Find the position of this character in `shiftedAlphabet`. Remember 'A' is represented 65 in ASCII code.

So subtracting 65 from the current character will give its position.

- Copy the character from `shiftedAlphabet` to `cipherText` using this position.

- Do not forget to return `ciphertext` to the `main()` function.

4. After the function call in `main()`, display the content of both `plaintext` and `ciphertext` using `cout`.
5. Run the program and observe its behaviour:



```
Microsoft Visual Studio Debug Console
Enter a MESSAGE to encrypt (Use UPPERCASE Characters):
UNIVERSITY
Enter the KEY (Use a NUMERIC value):
3

ORIGINAL ALPHABET : ABCDEFGHIJKLMNOPQRSTUVWXYZ
SHIFTED ALPHABET : DEFGHIJKLMNOPQRSTUVWXYZABC

PLAINTEXT : UNIVERSITY
CIPHERTEXT : XQLYHUVLWB

C:\Users\785317\source\repos\CaesarCipher\Debug\CaesarCipher.exe (process 21064) exited with code 0.
Press any key to close this window . . .
```

IV. DESIGNING DECRYPTION FUNCTION

We have developed a Caesar encryption system. Now we need the counterpart decryption system which you are required to finish the Caesar Cipher suite.

1. After decryption, need to ask the user to enter the ciphertext and the key as usual.
Use the same key and the same ciphertext generated in this Lab exercise.
You can use the code to read the plaintext and the key by changing `plainText` to `cipherText`.
2. Then call a function named `decryptMessage()` with appropriate parameters in `main()`.
This function should take `cipherText` and `shiftedAlphabet` as inputs and then should return the decrypted plaintext named as `decryptedPlainText`.
TIP: The variable `decryptedPlainText` should be defined in the `main()`.

The reversing of cipher character to original character needs a careful design. You can use the `find()` function of `shiftedAlphabet` to find out the first occurrence of the character in `shiftedAlphabet` specified by its arguments. The details of the `find()` function, as well as some examples, can be found in the following link:

<https://www.cplusplus.com/reference/string/string/find/>

3. Implement `decryptMessage()` outside of `main()`.
Where should you implement it?
Think of the opposite steps we used in the class exercise for `encryptMessage()`. Step by step reverse what we did.
4. Display `decryptedPlainText` in `main()` using `cout`.

Is it the same plaintext used to generate the ciphertext?

QUESTIONS:

1. Is this implementation complete? If not, what functionalities are missing considering encrypting a document or a text file?
2. Is this implementation effective? How can we improve the implementation?